

Analysis: Code Sequence

The Problem

Normally when you count in base 2, each bit is worth a fixed amount: {..., 8, 4, 2, 1}. So when you count 0000, 0001, 0010, 0011, and so on, you get the sequence [0, 1, 2, 3, ...]. In this problem, we ask you to consider what would happen if the bits were worth hidden amounts, "bit values," specified by us. For example, if the bits were worth {..., 200, 10, 1}, then you could get the sequence [0, 1, 10, 11, 200, 201, ...].

What this problem asks you to do is find the next number in the generated sequence. To make things more difficult, we don't necessarily start at 0000 or 0001, so what may seem obvious:

```
[1, 10, 11, 200, 201, ?, ...]
```

...may not be. This could have come from the bit values {..., 200, 10, 1}, which would give **210**; but it could also have come from bit values {..., -1180, 1190, 990, 190, 1}. If we start counting from 10000, that gives:

```
[-1180, -1179, -990, -989, -190, -189, 0, 1, 10, 11, 200, 201, 1000]
```

The Solution

So how do we solve the problem? Let's have a look at the generated sequence above, and look for interesting features. One thing that may strike you is that every second number is 1 higher than the last, so let's look at the sequence of differences between adjacent numbers:

```
(1, 189, 1, 799, 1, 189, 1, 9, 1, 189, 1, 799)
```

The differences of 1 make a lot of sense: every *second* increase in a binary number just changes the lowest bit from 0 to 1. So if your sequence of differences looks like (x, a, x, b), or (a, x, b), then the next difference must be x. Figuring that out should let you solve the small input.

But how far can we take this? Let's eliminate all the times only the lowest bit was changed. This leaves us with the following sequence of differences:

```
(189, 799, 189, 9, 189, 799)
```

Every second number there is a 189. That makes sense too: every *fourth* increase in a binary number is the same, changing the lowest two bits from 01 to 10. Likewise, every *eighth* increase changes the lowest three bits from 011 to 100.

So where does this leave us? Looking for repetition; seeing if we can use it; and if not, eliminating it. For example, if our generated sequence is:

```
[0, 1, 3, 4, 7, 8, 17, 18, 21, 22, 24, 25]
```

Our sequence of differences is:

```
(1, 2, 1, 3, 1, 9, 1, 3, 1, 2, 1)
```

Since the sequence ends with the repeated number, 1, we don't know yet what comes next. Eliminating the 1s, we arrive at:

(2, 3, 9, 3, 2)

Aha! The next difference must be 3, and the answer is 28. If that hadn't worked, we would have kept going recursively until we found the answer.

But what if the sequence of differences is:

(1, 2, 1, 2, 1)

Here, we can't tell if the lowest bit was changing for a difference of 1 or a difference of 2, so we can't say for sure what's next. We (eventually) arrive at the same problem in all of the following sequences of differences:

(1, 2)

(3)

(4, 3, 4)

(5, 4, 5, 3, 5, 4, 5)

(6, 5, 6, 4, 6, 5, 6, 3, 6, 5, 6, 4, 6, 5, 6)

Here it's IMPOSSIBLE to find the answer: that (1, 2) could be (1, 2, 1) or (1, 2, 10, 2). (4, 3, 4) could have come from:

10001 10010 10011 10100 10101

(4, 3, 4, **3**)

or from:

00100 00101 00110 00111 01000

(4, 3, 4, **?**)

There's no way to tell where you are in the range [0, 1000000000], so you could be about to hit a whole new bit and you wouldn't know it.

Comments

When we add up the values of our bits, we work modulo 10007. I used negative numbers earlier because the concept is well-defined in mod space: "-1180" mod 10007 is 8827, which is to say that adding 8827 to anything higher than 1180 is the same as subtracting 1180 from it.

One of the nice things about Code Jam is that you can test your assumptions. When writing the input generator for this problem, I asserted that the difference must look like [x, a, x, b, x, c, ...] or [a, x, b, x, c, x, ...]. This let me catch a bug in my code: I wasn't taking the differences mod 10007! In a live contest, I would have had time to fix that and submit.

Finally, there's one limit that's a red herring: that the sequence can't go past 10^9 . It isn't hard to convince yourself that the restriction is meaningless: any sequence of length 1000 can't affect more than 11 separate bits, so it doesn't matter if the highest bit is #13 or #32.

Analysis: King

The problem corresponds to a relatively unknown game named Slither. This game was popularized by Martin Gardner in one of his columns written for Scientific American. It wasn't solved then, but eventually William Anderson developed a solution. Of course Code Jam participants are much smarter, so we thought the problem would easily get solved in two hours.

I'll explain first the solution for a simpler variant of the problem. Suppose we have a 3×3 board with the king being in one of the corner cells. Now let's cover the rest of the board with dominoes of size 2×1 . This tiling of the board shows us a winning strategy for the second player. Each time the first player moves to one cell of a domino, the second player will move to the other cell of that domino. Since the second player can always make a move after the first player moves, the second player has a winning strategy.

The general solution goes along the same lines: the domino tiling corresponds to a maximum matching. We think of the board as a graph where the cells are the nodes, and neighboring cells have edges between them. The dominoes, then, correspond to edges in the maximum matching.

The solution is given by the following theorem: *"The first player has a winning strategy if and only if all maximum matchings contain the king's node"*.

This means that no alternating path starting from the king's node ends in a vertex that is not in the matching, because if it ended in a unmatched vertex we could construct another matching with the same number of edges that doesn't use the king's node. Thus the first player could always move along a matched edge.

If there is a maximum matching that doesn't contain the king's node, then in this graph every alternating path that starts from the king's node ends in a matched node, otherwise we could increase the size of the matching which contradicts the assumption that the matching is maximum. Thus now the second player can always move in the second node of the matching edges. So the second player has a winning strategy.

The graph in our problem is not bipartite so we can't use the standard algorithm for bipartite matching. Instead we use a solution that combines brute force and dynamic programming. Our solution will be exponential rather than polynomial, but the size of the problem is small so we can afford it. We go row by row from left to right, and for each cell (i, j) and each subset S of $\{0, 1, \dots, n-1\}$ we compute the best matching that has the following nodes matched already: nodes on the active line ($[i][0..j]$, $[i-1][j+1 \dots n-1]$) that correspond to the numbers in S , and matched nodes from before the active line.

Here's Derek Kisman's code that implements this solution:

```
#include <algorithm>
#include <iostream>
using namespace std;

int gx, gy;
char g[15][15];

char memo[15][15][1<<16];
char doit(int x, int y, int b) {
    if (x == gx) {
```

```

    x = 0;
    if (++y == gy) return 0;
}
char& ret = memo[x][y][b];
if (ret != -1) return ret;
int b2 = (b<<1) & ((1<<(gx+1))-1);
if (g[y][x] != '.') {
    ret = doit(x+1, y, b2);
} else {
    ret = doit(x+1, y, b2+1);
    if (x && (b&1)) ret >?= 1 + doit(x+1, y, b2-2);
    if (x && (b&(1<<gx)))
        ret >?= 1 + doit(x+1, y, b2);
    if (b&(1<<(gx-1)))
        ret >?= 1 + doit(x+1, y, b2-(1<<gx));
    if (x < gx-1 && (b&(1<<(gx-2))))
        ret >?= 1 + doit(x+1, y, b2-(1<<(gx-1)));
}
return ret;
}

main() {
    int N, prob=1;
    for (cin >> N; N--;) {
        cin >> gy >> gx;
        int kx, ky;
        for (int y = 0; y < gy; y++)
            for (int x = 0; x < gx; x++) {
                cin >> g[y][x];
                if (g[y][x] == 'K') {kx = x; ky = y;}
            }
        memset(memo, -1, sizeof(memo));
        int m1 = doit(0, 0, 0);
        g[ky][kx] = '.';
        memset(memo, -1, sizeof(memo));
        int m2 = doit(0, 0, 0);
        cout << "Case #" << prob++
              << ": " << ((m2 > m1) ? 'A' : 'B')
              << endl;
    }
}

```

Analysis: Mixing Bowls

Looking at the restrictions on the recipe, it should be obvious that the recipes form a tree. It is also clear that basic ingredients can be ignored: You just throw them into whatever bowl you're currently working on.

Lets consider a recipe with k mixtures $m_1, \dots, m_k$. How many bowls do we need to prepare this recipe? It does not make sense to work on more than one mixture at a time, so let's try preparing the mixtures in the given order.

We'll need b_1 bowls to prepare the first mixture. For the second mixture, we can reuse these bowls, except for one bowl that we need to hold the first mixture. While preparing the third mixture (which requires b_3 bowls in itself) we will need two additional bowls to hold the first two mixtures, and so on. Finally, once we have prepared all the mixtures, they sit in k different bowls, and we'll need one additional bowl to put everything together, and finish the recipe.

How many bowls did we use in total? Keeping in mind that we can reuse bowls from the previous step (except for the bowls to hold finished mixtures), we will need $b = \max(b_i + i - 1)$ bowls to prepare all mixtures, for a total of $\max(b, k + 1)$ to put everything together.

Looking at this formula, it is obvious that the number of bowls will be minimized if we prepare the mixtures requiring the most bowls first. Thus, our algorithm will first invoke itself recursively for every ingredient, sort the results in descending order, and then use the formula given above to return a result.

Analysis: Test Passing Probability

The Problem

There are 4^Q points in the space, each of which contains an *answer combination* -- one answer for each of the Q questions. Because the answers to the questions are independent of each other, the probability of success at each point can be computed simply as the product of the probabilities from each problem.

So the simple mathematical model involves 4^Q points, each with a probability value. There is a single, hidden good point among the 4^Q points; you may pick up to M points; and you win the game if one of them is the good point. After you select a point, you learn if it is good or not, but that does not change the relative probabilities of the remaining points. The best strategy is simply looking at the points in the order of their probability. Your task in this problem is to compute the summation of the highest (up to) M among the 4^Q values.

Solutions

While Q looks moderate, 4^Q steps of computation can be a huge task that is probably not within your computer's capacity (or any computer's). The important restriction in this problem is the fact that M is also moderate, and we can compute the highest M values one by one.

One solution uses a priority queue L that keeps all the answer combination candidates for the *next* highest value -- after we computed the first k values. A bit of thought should convince you that the next-highest (the $k+1^{\text{th}}$) combination must be gotten by taking one of the first k combinations, and moving the answer to one of the questions one step "worse", to the next-most-probable answer. So in each step we can get the first element S (the one with the highest probability) from L , then add (up to) Q new candidates back to L . Each of the new candidates is created by moving one answer in S one step worse. The size of L will never exceed MQ , and the time complexity is $O(MQ \log MQ)$.

Another solution has an even simpler implementation: the idea is to add the questions one by one. For the first k questions, there are 4^k possible solution combinations, each with its own probability of answering all of the first k questions correctly. When we add the $k+1^{\text{th}}$ question, the new set of possible solution combinations, along with its probabilities, can be calculated by taking each of the 4^k previous values and multiplying it by the four probabilities for each answer to question $k+1$. We can easily compute all of these values and truncate to the top M , then repeat the process for the remaining questions. The truncation is $O(M \log M)$, and this process is repeated Q times, for a total time complexity of $O(QM \log M)$.

More information

[Discrete Probability](#) - [Joint Distribution](#)